

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Studying Learned Features in Language Models

by **Neel Nanda** 19th Jan 2023

This is the final post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#), then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Motivation

Motivating Paper: [Softmax Linear Units \(SoLU\)](#), [Multimodal Neurons](#)

*To accompany this post, I've created a website called [Neuroscope](#) that displays the text that most activates each neuron in some language models - **check it out!***

This section contains a lot of details and thoughts on thinking about neurons and learned features and relationship to the surrounding literature. If you get bored, feel free to just skip to exploring and looking for interesting neurons in [Neuroscope](#)

MLPs represent $\frac{2}{3}$ of the parameters in a transformer, yet we really don't understand them very well. Based on our knowledge of image models, my best guess is that [models learn to represent features](#), properties of the input, with [different directions corresponding to different features](#). Early layers learn simple features that are basic functions of the input, like edges and corners, and these are iteratively refined and built up into more sophisticated features, like angles and curves and circles. Language models are messier than those image

models, since they have attention layers and a residual stream, but our guess is that analogous features are generally computed in MLP layers.

But there's a lot of holes and confusions in this framework, and I'd love to have these filled in! **How true actually is this in practice? Do features correspond to neurons vs arbitrary directions?** What kinds of features get learned, and where do they occur in a model? What features do we see in small models vs large ones vs enormous ones? What kinds of things are natural for a language model to express vs extremely hard and convoluted? What are the ways our intuitions will trip us up here?

Issues like [polysematicity](#) and [superposition](#) make it difficult to actually reverse engineer specific neurons, and it seems clear that the model can learn features that do not correspond to specific neurons. But even if we relax our standards of rigour and just want to understand what features are present at all, we don't know much about what role these layers actually serve in the model! There's been a fair amount of work studying this for BERT, but very little on studying this in generative language models, especially large ones!

There's a bunch of angles to make progress on this, and I'm excited to see a diverse range of approaches! But here I'm going to focus on the somewhat unreliable but useful technique of looking at **max activating dataset examples**: running the model across a bunch of data and for each neuron tracking the texts that most activated it. If we see a consistent pattern (or patterns) in these texts we can infer that the model is detecting the feature represented by that pattern. To emphasise, **a consistent pattern in a neuron's examples does not mean the neuron *only* represents that feature**. But it can be pretty good evidence that the feature is represented in the model, and that that neuron responds strongly to it!

To help you explore what's inside a language model, I've created a tool called **Neuroscope** which displays the max activating dataset examples for a range of language models. Each neuron has its own page, and there are some pretty weird ones in there! For example, take [neuron 456 in my one layer SoLU model](#) - this activates on the word " the " after " love ", on what *seems* to be the comments section of cutesy arts and crafts blogs.

matching border). It really gave this a fun energy.
Such a cute job with this adorable stamp! Gorgeous bow!
Love that cute retro stamp. Great card!
Beautiful CAS design. Fantastic use of colour. I love the little pops of
orange - very eye catching.

<|BOS|> a gorgeous card and this is my favorite sentiment stamp!!!
Beautiful card!! I love the holly and the kraft paper with the red

honeycomb paper !!
 Beautiful card! Really love all the extra details! Have enjoyed the little hops!
 Love the design on your card. The paper is perfect and love the way you used the sprinkles!!
 Pretty card. I love the lil bees.
 i am digging all of

One vision for an accessible, beginner-friendly project here is **to just explore Neuroscope** - look through a bunch of neurons and see if you can spot any interesting patterns. And once you've found one, **load in that neuron to an interactive interface**, and feed in a range of text to refine your understanding of what does (and doesn't!) activate the neuron. Another vision is to **make predictions about what features the model *should* want to represent**, and then to feed in a bunch of examples and look for neurons that consistently activate (and to then look these up in neuroscope), or to train a probe analysing whether the feature can be recovered from the residual stream. The latter doesn't require features to be at all neuron-aligned, but I'm particularly excited about the former, because it allows you to be **surprised** by what you find inside the model - you aren't just anchored to what features you predict will be there!

Ultimately, we care about rigorously reverse engineering what features and circuits the model has learned, and I don't expect these projects to achieve that level of rigour. So, why care about any of this? I break this down in a few ways:

- I want a clearer understanding of **what features are inside models**. This will help build good foundations and intuitions for thinking about what models actually do and what circuits we might expect to find.
 - Under this perspective, neurons are just a means to an end - we don't care much about whether they're monosemantic or polysemantic. But we *do* care a lot about whether we've identified the true feature, or are missing some key detail, and it's important to [use an interactive interface](#) to actually verify!
- This serves as a good basis for future projects, where you then go and actually **try to reverse engineer the features you find!** I am very excited to see projects trying to reverse engineer specific features or neurons, and drilling down into how much they actually align in practice.
 - Under this perspective, we care a lot about neurons, and understanding how much a good project looks like reverse-engineering a specific neuron vs a representation distributed across several neurons vs an arbitrary direction in activation space.
 - See [the case for analysing toy language models](#)° or [circuits in the wild](#)° for more.

- Understanding **how neurons fit into the bigger picture** of what's going on inside a language model - how often are they monosemantic? Are “monosemantic” neurons monosemantic throughout their entire range, or only at the tail end? Do features correspond to a small handful of neurons, or are they highly diffuse? How often do neurons play a role that is *not* well thought of as representing features, like [memory management](#)?
 - This links heavily to [polysemanticity and superposition](#)°

Finally, I just think that this is a solid thing to play around with if you're new to the field - **it's incredibly easy to get started staring at neurons**, and can hopefully be fun and give inspiration for more involved projects! (Though other projects may appeal to different tastes)

Related Work

This topic overlaps much more with the wider interpretability literature than the other posts, so this section is intended as a light literature review - I hope it's helpful, but make no claims that it's covered everything important, nor that I've accurately summarised each work! Understanding this isn't essential for basic projects to do with identifying interesting neurons and features, but may be useful for more ambitious projects.

The paper that most inspired this post was **Softmax Linear Units (SoLU)**

(disclaimer: I was involved in the paper) - my favourite section is the qualitative exploration of [what neurons they found](#). I summarise the paper [here](#). Some of my favourite features:

A Base64 neuron - internet text is often written in base 64 (eg shortened links), and in a 1 Layer model they found a neuron that seemed to detect this. I think of these as **context neurons** - neurons which activate on a range of text that all share some common feature (eg `is_newspaper_headline`, `is_python_comment`, etc).

BASE64 NEURON (IN ONE-LAYER MODEL)

Dataset Examples

<EOT>8efMXtnduFiZVfa
rzKbO9NszTKzaVMq51cOISAE0otA3QRy/IOzlU+bPRNinePI95g98bBZx5fTC5BqJgSEQGkE6iKQ
0x+azVlqduRYacXVpkybbLbrebPOa6otoXxCQAikBYGaCQTyuPIOs5MfROdC1z+Y/erfRSLRISpN
QiAeBGqahaHbQuQRJXngJvrQi/6sCotYpXW5yay2ifebRcckxRGoiUAY84ii21KsKuh9ZE0xIGwc
Y8W4Bx98MO8sJ+3ChQvd0g2kVRK+zs4yD/4r7T/96U/dwleVyrVSOiVw8Re1zJkzx3bs2BG12pbQ

"I'm not going to hold back." Brantley takes important step in comeback:
<https://t.co/O88CkxPQhw> pic.twitter.com/UAYcx7cPqk — Jordan Bastian (@MLBastian)
March 17, 2016

Clear acrylic pipe for filtration: <https://amzn.to/30kNQGg>
Clear acrylic pipe products: <https://amzn.to/30hJj7B>
Protein Skimmers of choice: <https://amzn.to/2LDu1Xn>
Finnex LED Aquarium light: <https://amzn.to/2wOlbie>

How is "Economic security for all who are unable or unwilling to work" vague? Progressives
are pretty clear about what "economic security" means to them.<https://t.co/CaHliQxszw>
pic.twitter.com/GfxG7ZK8D4 — Jeryl Bier (@JerylBier) February 9, 2019

Logit Weights

+0.17 'Wl'	-0.60 ' section'
+0.15 'zA'	-0.60 ' segment'
+0.14 'mème'	-0.60 ' of'
+0.14 'Rp'	-0.61 ' exam'
+0.13 'Oi'	-0.61 ' hatch'
+0.13 'Cc'	-0.61 ' dusk'
+0.13 'Tk'	-0.61 ' eye'
+0.13 'Hg'	-0.62 ' count'
+0.13 'Hz'	-0.62 ' time'
+0.12 'mV'	-0.62 'levance'
+0.12 'ZX'	-0.62 ' cent'
+0.12 'bW'	-0.63 ' circumference'
+0.12 'GQ'	-0.63 ' balances'
+0.12 'Cb'	-0.63 ' operand'
+0.12 'Nz'	-0.64 ' volumes'
+0.12 'rU'	-0.64 ' quadrant'
+0.12 'fq'	-0.65 ' surface'
+0.12 '+/'	-0.66 ' volume'
+0.12 'Dt'	-0.70 ' end'
+0.12 'Jy'	-0.72 ' compartment'

De-tokenization neurons - early layer neurons that convert the raw token inputs into a format more useful to the model. These feel vaguely analogous to **sensory neurons** in biology - they convert the “raw input data” into a more useful format. Eg merging pairs of tokens that go together: “Donald| Trump”, “|left”, “social| security”. A particularly fun one are *families* of neurons, where each neuron responds to the same token, but in a specific context. Eg a family which activates on “die” in different languages - the token has a fixed representation, but intuitively the “die” *should* mean different things in different languages:

“DIE” IN AFRIKAANS NEURON

Dataset Example

"Die wassemiekollit, Pa weet, die cowboy wat in die prent speel, die held wat die kroeks
platskiet, die gesteelde goud terugkry – én die mooiste meisie."

“DIE” IN GERMAN NEURON

Dataset Example

Die Helfer wollten unbedingt verhindern, dass das schädliche Klebemittel in die Kläranlage
fließt und dort die biologische Reinigungsstufe beeinträchtigt, wie Tiefbauamtsleiter Peter

“DIE” IN DUTCH NEURON

Dataset Example

Vergissen ze zich? Rida rent zo hard als hij kan langs de woonboten, richting de Range
Rover. Hij hoort meer knallen. Het zijn de schoten die even verderop Youssef het leven
kosten.

Re-tokenization neurons - late layer neurons that fire in the middle of multi-token words to output the correct next token. Eg “n|applies” is a 3 token word, so once the model has

concluded that the next word is nappies, it needs to actually take the actions of outputting “app” and “ies”. These feel analogous to **motor neurons** in biology - the model has figured out that the answer is nappies as represented in some conceptual space, and now needs to explicitly convert this to the actions of outputting the correct tokens.

N->APPIES

Dataset Examples

The room smelt of warm milk, wet **n**appies and lanoline.

containing disposable **n**appies.

Top Logit Weights

+0.32 'app'

ST->RAGGLER

Dataset Examples

a few **s**tragglers remaining behind,

Now they held only a few last-minute **s**tragglers

Top Logit Weights

+0.31 'rag'

“Sophisticated neurons” that seem to be doing some more complex processing in middle layers of larger models. Eg a “numbers that implicitly refer to groups of people” neuron

NUMBER (IMPLICITLY OF PEOPLE)

Dataset Examples

The main banquet room can seat up to **150** guests. This room features neutral decor and the large fireplace adds a warm glow for spring, fall and winter events. The floor to ceiling windows overlook the 9th and 18th holes of our championship golf course.

Star Resorts. In addition to standard hotel rooms, the All-Star Music and Art of Animation Resorts offer two-room Family Suites that can sleep as **many** as **six** and provide kitchenettes.

The Legacy Chapel can accommodate up to **70** guests. The Cherish Chapel can accommodate up to **45** guests. The outdoor Terraza overlooks the pool and can accommodate **100** guests.

business in a small garage to become the world's largest manufacturer of "build-it-yourself" component car kits. They employ a full-time crew of about **40** people, and are located in Wareham, Massachusetts (about an hour south of Boston). They make their products right

This work helped significantly clarify my intuition on what’s actually going on inside generative language models, in particular the framework that early layer neurons often act as **sensory neurons**, taking in the raw token data and piecing it together into more useful

representations, middle layers do the actual conceptual processing, and later layer neurons often act as **motor neurons** which convert this conceptual understanding into the actions with high loss.

Note that the focus of the paper was on how the [SoLU activation function](#) they introduced seemed to make neurons more interpretable (compared to GELU activations), done via the metric of what fraction of the neurons had a consistent pattern in the max activating dataset examples. In my opinion, this showed much more convincingly that the neurons *strongly* react to that feature, than that the neurons *only* react to that feature.

There's been some other cool work studying individual neurons:

- There are lots of great image circuits papers doing high-level studies of many neurons (though alas in image models), most notably [Multimodal Neurons](#) (in CLIP) and [An Overview of Early Vision in InceptionV1](#).
 - [Curve Circuits](#) is a great deep dive into a specific neuron family, and a good model for what *really* understanding a neuron can look like..
 - [Multimodal neurons](#) has numerous case studies where they dive deeper into specific neurons, which may be good models to follow.
- [Transformer Feed-Forward Layers Are Key-Value Memories](#) looks at the max activating dataset examples of neurons in GPT-2, and often observes patterns. And has some exploration of automatically detecting the neurons systematically boosting the logit for the correct next token (ie, where some of the neuron's effect is by directly impacting the logits).
 - The focus of the paper is on the framing of neurons as looking up some direction in the residual stream (key) and outputting another vector if that key direction is present (value), and they don't focus on cataloguing and categorising the neurons they find (though discuss some in Table 1).
 - Note that key and value mean *completely different* things to key and value in attention layers!
- [Identifying and controlling important neurons in neural machine translation](#) is a fun older NLP paper studying LSTMs trained to translate languages, with a bunch of analysis of specific neurons in [Appendix C](#)
- **Pitfall:** [The Interpretability Illusion](#) is a great paper pointing out **the pitfalls of just studying max activating dataset examples**.
 - They found a neuron in BERT which looked monosemantic when dataset examples were studied on different datasets, but where *different* patterns

occurred - on trivia questions, sentences to do with song lyrics, on Wikipedia, historical events/sentences beginning with dates, and on books, taking objects out of a container. (and say that these similar issues apply to other neurons too)

- This can be partially addressed by always taking the max over the data distribution that the model was trained on, but in my opinion indicates a genuine issue. My interpretation is that each feature is represented by a direction in activation space, with a significant coordinate along that neuron. But that the size of these coefficients varies between features. Each dataset only has some features present, and so only the feature with the highest coefficient *that occurs* shows up.

There's a range of broader work studying learned features and representations in models:

- **Literature reviews:**

- [Towards Transparent AI](#) provides a good literature review in section 3
- [A Primer in BERTology](#) reviews the field of BERTology, which included a significant amount of study of what syntactic/linguistic knowledge is represented in BERT
- A notable subfield is the study of probing - in the simplest case of a linear probe, this looks like taking a dataset of examples labelled by whether or not they contain a feature, and training a linear function on a network activation to extract that feature.
 - [A good literature review](#) from Yonatan Belinkov.
 - [Visualizing and Measuring the Geometry of BERT](#) is a cool study on what linguistic features are represented in BERT, geometric analyses, and how these break down into subspaces
 - [Accompanying blog post](#)
 - [BERT Rediscovered the Classic NLP Pipeline](#) is a classic study of how the internals of BERT can be somewhat mapped on to standard wisdom about how to understand and process language
 - [Testing with Concept Activation Vectors](#) (TCAV) is an interesting study (in image models) which both trains a linear probe to find the direction of an activation corresponding to a concept, *and* takes the derivative of some metric (eg the loss) with respect to that activation, and compares these two directions to approximate how important that concept was to a classification.
 - **Pitfalls:** [On the Pitfalls of Analyzing Individual Neurons in Language Models](#) gives some critiques and limitations of common approaches to using probing to study individual neurons.

- **Attribution methods** are ways of “attributing” to different scalar activations how important they are for producing some metric (eg model loss or probability of the correct classification). Intuitively, this is about credit allocation - giving each scalar a score such that we account for its unique contribution without double-counting. The focus is normally on attributing each scalar in the *input* to the output, but it can be easily extended to intermediate activations like neuron activations.
 - **Integrated Gradients** is a particularly well-known and principled method, which attributes each scalar element of the input’s effect on a metric by varying the input, taking the partial derivative of the metric with respect to each input element, and integrating those. I try to explain in more depth in [my mech interp explainer](#). Note that integrated gradients work best when we imagine varying several neurons, and want to assign credit to each (eg scoring each neuron in an MLP layer).
- **Feature visualization** is a really useful technique to interpret neurons in image models - it synthetically generates an image that maximally activates that neuron, and these are often interpretable pictures, which can act as a “variable name” for the neuron. Sadly, this is very specific to the continuous input space of images, and doesn’t seem to work for the discrete input of language models (but I’d love to see someone prove me wrong!).
 - Here’s [a good write up](#) from some researchers who tried valiantly to extend this to BERT with tricks like Gumbel-Softmax (to pretend that discrete inputs are continuous) and mostly failed. (I wish people would write up negative results like this more often!)

Tips

- First, recall how [neurons in a transformer](#) work. They’re the hidden state of an [MLP layer](#), where there’s a linear map W_{in} from the residual stream to a hidden state, an activation function (in the Neuroscope models, either [GELU](#) or [SoLU](#)), and a linear map W_{out} back to the residual stream.
 - Each neuron i is a specific element of the hidden state, and so is associated with a single input vector $w_{in} = W_{in}[:, i]$ and single output vector $w_{out} = W_{out}[i, :]$. Both vectors are directions in the residual stream!
 - The output of the neuron for a given residual stream value r is

$$r \rightarrow w_{out} Act(w_{in} \cdot r)$$

- The output of the MLP layer is the sum of the output of each neuron.

- It is very easy to be wrong about the feature you've found, especially if this is solely from intuition based pattern recognition.
 - It's easy to form hypotheses that aren't specific enough (eg, thinking it's a cutlery neuron when it's actually a fork neuron)
 - Or hypotheses that are *too* specific (eg thinking it's a neuron about romance novels, when it's actually a neuron about fiction and your dataset mostly contained romance fiction)
 - The way to manage both of these is to try to create a diverse range of examples that are more or less specific and run these through the model ([using the interactive neuroscope](#)). You want to ensure you *both* have examples that you predict will activate the neuron/direction, and examples that you predict will *not* activate it.
 - **The goal is to try to falsify your hypothesis**
 - Eg try deleting words, adding random noise, replacing them with synonyms, etc.
 - Example: Let's say we have an Eiffel Tower neuron - the text tokenizes as "E|iff|el| Tower", and the neuron activates on " Tower". Naively, it should be checking for the presence of " E" AND "iff" AND "el" AND " Tower". But this is pretty overdetermined, and it could plausibly just be looking for, say, "E|iff|_| Tower".
 - A sensible approach to editing the text could easily try the obvious thing of deleting "Eiffel" or "Tower", and see that it breaks the neuron, and of deleting all prior words and see that it does nothing. And thus conclude that it's an "Eiffel Tower" neuron, without checking the tokens!
 - Note that this is a somewhat contrived example - it's reasonable to not care about an "E|iff|_| Tower" neuron vs a "E|iff|el| Tower" neuron, but this kind of subtlety can underlie more significant effects.
 - Some simple ideas to refine your understanding of the input needed to activate a neuron:
 - Add noise to the embeddings of each token in turn, and look for which tokens matter the most.
 - Resample each token in turn (replace with a randomly chosen other token)
 - Add in random tokens *between* tokens in the input, and see if this breaks anything
 - Truncate the input (only feed in the final k tokens), and see how small k needs to get before the neuron stops activating much.

- Replace important words with synonyms, opposites, or other vaguely related words.
- MLPs serve a very different role to attention in the model - they cannot move things between positions, but they can do a lot more computation at the current position. A rough brainstorm of the kind of things they can do (Note - the below is a brainstorm of what MLPs *should* be able to do in theory, and is not particularly validated by actual empirical data):
 - **Boolean AND:** If feature A *and* feature B are present, produce feature C
 - This is particularly important for eg trigram neurons, like “ice cream” -> “sundae”.
 - This was a carefully chosen example, because “ice|_” and “_| cream” both can mean very different things from ice cream, so the model *needs* to be doing an AND.
 - In contrast, eg “Barack| Obama” may not even need a neuron, since either token on its own should be sufficient.
 - On the other hand, you want to treat text like “Barack Obama” and “The Obama Administration” pretty differently!
 - **Transforming an existing 1D activation** - for a pre-existing feature/direction, our goal is not to change the feature, just to apply a non-linear transformation
 - This can be particularly useful when combined with other parts of the model, eg attention.
 - For example, let’s say a model wants to count the number of times “Section” has appeared in the text so far.
 - This can be counted with an attention head whose QK circuit gives 1 on Section, 1 on BOS and -inf on everything else, and whose OV circuit is $1 \times \text{the “is_section” direction on Section, } 0 \text{ on everything else.}$
 - The output of this head is $(n/(n+1)) \times \text{“is_section”}$.
 - We want this to be, eg, $n \times \text{“is_section”}$, and this is impossible to do linearly, but easy to approximate with a set of neurons
 - **Sharpening** - Where we take an activation that is varying fairly smoothly, and adjust it to something more discrete.
 - Example: Rather than counting occurrences of “Section”, mapping it to a “is at least 3 occurrences of section”
 - **Saturating** - When we want to *stop* an activation increasing beyond a certain point. Essentially $x \rightarrow \min(x, \text{threshold})$. A form of sharpening.

- Example: “is_python_code” is an important feature to have, and significantly changes unigram frequencies, the features to look for, etc. And the natural way to compute this is by having attention heads look for Python-specific motifs: for, while, yield, whitespace, etc.
- But it’s also fairly bimodal - it shouldn’t be that hard to figure out if you’re in Python code or not! The *amount* of evidence we get is likely to vary a bunch, but we want to act basically the same if there is a lot of evidence or a ton of evidence.
- So if we want the “is_python_code” feature to boost the probabilities of Python-specific next tokens, we want this feature to saturate.
- **Thresholding** - The opposite of saturating, when we *only* want feature B if feature A crosses a certain threshold. Essentially $x \rightarrow \max(x, \text{threshold})$. A form of sharpening.
 - Example: We might only want to activate certain circuits if we’re confident something is Python code. We can set this by thresholding our “is Python” evidence at a certain level.
- **Lookup table** - as argued in [Geva et al](#), they can implement a lookup - if feature A is present $\rightarrow$ feature B
 - Note that this is surprisingly subtle! If the model wants, every time a circuit looks for feature B, it could just look for feature A instead. So the neuron needs to be doing *something* more sophisticated. Possible ideas:
 - The neuron is also acting as an AND - to lookup “Bill Gates” $\rightarrow$ “founded Microsoft”, it actually has the two inputs “prev token == Bill” and “current token == Gates” (implemented by having w_{in} be the sum of the two features)
 - The neuron acts as a ReLU - if feature A is positive we release feature B, but if the feature A direction is *negative* (eg it’s being represented with superposition or there’s actively evidence against it being present), we do not release negative feature B
 - There are other ways to calculate feature B, and the neuron is showing that feature A is *equivalent* to the other ways. Eg “Eiffel Tower” $\rightarrow$ “is located in Paris” makes sense, if there are other neurons for different French landmarks, which all output to the same “is located in Paris” direction.
- **Non-Feature Based Operations** - roles the neuron may serve in the transformer, other than directly computing or processing features

- **Memory management** - by default the residual stream is the sum of *all* previous layers. But sometimes previous layer outputs get used by the next layer and then are no longer needed. Neurons are a natural way to delete this.
 - [A Mathematical Framework](#) observed that sometimes neurons had a high negative cosine similarity between w_{in} and w_{out} , suggesting that this could be their role.
- **Signal boosting** - because the residual stream gets larger (in norm) as the layers progress, any given feature becomes a smaller and smaller fraction as the model progresses. For important features, eg positional embeddings, the model may want to promote these to remain a significant fraction of the residual stream
 - This is the inverse of memory management, and as such should be represented by neurons with high *positive* cosine similarity between w_{in} and w_{out}
- **Cleaning up superposition interference** - If the model is [using superposition](#) (compressing more than n features into an n -dimensional space), there will be [interference between features](#) - if feature A is present and feature B is not, but their directions are not orthogonal, it could *look* like feature B is present. Neurons are a natural way to deal with this kind of noise, as shown in [Toy Models of Superposition](#)
- When editing the input to the model, pay close attention to the tokenization! It's easy to make a minor edit to the text which completely changes the resulting tokens (especially the token position the model is on!)
 - It can be easier to split the input text into tokens (as strings, the method is `model.to_str_tokens` in TransformerLens), edit *that* split list, ensure each element of the list remains a single token, and convert it back to the input.
- If the model wants to do several things with the same “neuron input”, it can do all of them with a single neuron!
 - Example: It wants to learn a neuron for the trigrams “ice cream” -> “sundae” and -> “parlour” and -> “cone”. Naively, this should take three neurons, but we can easily have a single neuron, whose output vector is the *sum* of “predict sundae” and “predict parlour” and “predict cone”
 - In some sense, this is *just* a bigram neuron - the neuron feature “ice cream” can just be mapped by the unembed to “sundae”, and it can also be used by other layers to indicate that this position is the “cream” in ice cream.

- Example 2: The model wants to learn several facts about Bill Gates - “studied at Harvard”, “married Melinda French”, “founded Microsoft”, “is philanthropist”, etc. Since these are all activated by the same input, there can just be a single neuron that does *all* of them!
- One important consequence of this is that, just because a neuron’s output boosts the correct next logit, doesn’t mean that that’s *all* the neuron does.
- People often use “neuron” to refer to many different parts of a transformer. I *specifically* mean the hidden state of the MLP layers, after the activation function. I do *not* mean the residual stream, layer outputs, keys, queries or values, attention pattern, etc.
 - For SoLU I take it to mean the activation after the SoLU and before the LayerNorm (denoted as `hook_mid` in TransformerLens), but I’m uncertain about this, and think that post LayerNorm or pre-SoLU are also interesting to explore.
- If you want to use this as a base for a project to actually reverse engineer a specific neuron or feature, I highly recommend focusing on toy language models, especially one layer models - interpreting neurons is hard even in a one layer model, and gets much harder with each bit of added complexity!
- SoLU is likely to have more interpretable neurons, but also more of a headache to attempt to actually reverse-engineer than for GELU models. In GELU, the neuron activation is *solely* a function of the neuron value pre-activation. In SoLU, the softmax mixes things between neurons, and the LayerNorm mixes things further.
 - Note - mathematically, SoLU activations consist of

$$x \rightarrow x \odot \text{softmax}(x) \rightarrow \text{LayerNorm}(x \odot \text{softmax}(x))$$

Generally the first step is referred to as the SoLU activation.

- In particular, this means that the equation above is not correct:

$$r \rightarrow w_{out} \text{Act}(w_{in} \cdot r)$$

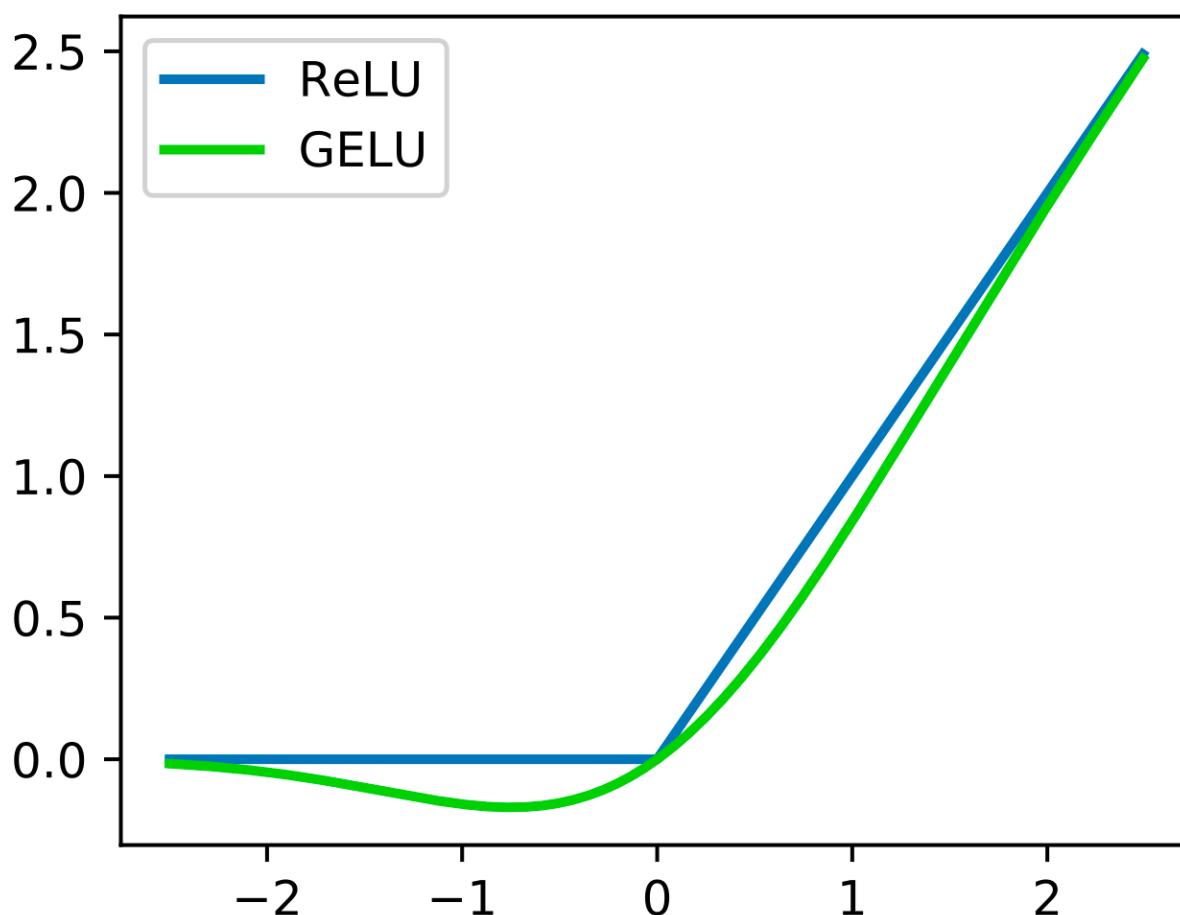
becomes

$$r \rightarrow w_{out} \text{Act}(W_{in} \cdot r)[i]$$

- The paper speculates that SoLU models are not actually doing less superposition, but rather that the model sometimes represents features with a single neuron, and sometimes as a highly diffuse representation across many neurons. When a feature is represented with many neurons, the output of the SoLU is small, and so the LayerNorm scales it up, allowing it to “smuggle through” superposition.

- If the embedding and unembedding are *not* tied, neurons should be a function of something more than *just* the current token, because the embedding is already an arbitrary lookup table - for each input token we can memorise an arbitrary vector.
 - Example: If we want a neuron that eg maps the token “Obama” to the “is politician” feature, then in theory the embedding could learn the sum of the “is Obama token” direction *and* the “is politician” direction.
 - Note that this does *not* apply if the embedding and unembedding are tied, ie $W_U = W_E^T$. The embeddings are tied in GPT-2, but not in any of my models (toy language models or SoLU models)
- Anecdotally, **the first MLP layer in GPT-2 acts as an extended embedding**. That is, future layers treat the token embedding as embed + MLP0, rather than just embed.
 - I am not particularly confident in this (and have mostly seen evidence in GPT-2 Small), and speculate that this is because GPT-2 has tied embedding and unembeddings, so the model is forced to do this to break the symmetry
 - But this is important if true, because that suggests that many of the neurons in that layer will be doing single-token function stuff. (And probably some neurons will be doing other things, and it'll be using the first attention layer too)
- When a neuron acts on one or several features, the **activation range** of those features matters a lot. Eg, whether they're a binary on or off feature (like current token == “the”) or a more continuous feature with lots of evidence for and against it (like “is end of sentence”).
 - For example, let's consider a neuron is doing AND of feature A and feature B,
 - If both of those are binary, this is easy. It can have w_{in} align with both features, and it can just apply a threshold between $\max(\text{Feature A}, \text{Feature B})$ and $\text{Feature A} + \text{Feature B}$.
 - But if both are continuous, this is much messier - a single neuron can't tell the difference between medium Feature A + medium Feature B vs high feature A + no feature B!
 - Further, it's much easier to do superposition with binary features rather than continuous ones, because there's less interference to account for (eg, [a model can memorise arbitrarily many orthogonal data points with just two dimensions](#)).
- **GELU vs ReLU** - Most of my intuition about neurons comes from thinking about ReLUs ($x \mapsto \max(x, 0)$), which were used in image models, and not from GELUs, which are used in language models. GELUs are *kind of* a smooth ReLU, and can do most of the same things. But they aren't the same, and models may exploit that.

Nonlinearities



-
- GELUs are *not* monotonic. Because there's a "bowl", $\text{GELU}(-2) > \text{GELU}(-1)$. This never happens with ReLU.
- Around the bowl, GELUs act more like a quadratic function than a linear function, which the model may exploit to get more out of them/
- It's easy to confuse re-tokenization neurons and bigram neurons - even if the model *didn't* calculate that the answer should be "n|applies" before the "n" token, once it's seen "n" and "app", figuring out "ies" is easy. You want to distinguish these by identifying whether the words *before* nappies matter.
- Remember that GPT-style models have causal attention! The neuron activation at position k can *only* be a function of the tokens up to position k . This means that the text after the high activation in a dataset example *cannot* be relevant to the model's computation.
- Models may use several neurons to represent a single feature, rather than either a single monosemantic neuron or a direction that's diffuse across many neurons. This is suggested by [Toy Models of Superposition](#)
 - One thing that's very annoying about this is that the neurons do not need to be in the same layer! Let's suppose a feature is represented by 5 neurons in layer 13. So

long as they aren't used by layer 14 but are only used by later layers (which is pretty normal for a transformer), we could just as easily have 3 of those neurons be in layer 14 instead.

- When looking for a neuron representing some feature, it's important to check that that feature is actually present at all!
 - One approach is to make a small dataset of examples with and without the feature and train a linear probe on the residual stream to detect the feature
 - Remember to have a test set!
 - A good sanity check is to randomise the feature labels, and check that your probe does *not* learn to predict the random labels on the test set.
 - If you think the neuron represents a feature, it can be cool to compare how well you can detect it with your probe vs just taking that neuron.
- Some features may be more efficiently computed with attention heads.
 - Example: The model will want to have an “is newspaper headline” feature on each token in a headline. But plausibly, rather than having a neuron which activates for any token *in* the headline, it's easier to have a neuron that activates on the *first* token to produce an “is start of headline” feature, or a token before (eg CNN), and an attention head that moves that feature to become an “is in headline” feature for any token in the headline.
- A big, important feature that you expect to be learned by the model may fail to be present because of **neuron splitting** - the model instead decides to break that feature down into many, more specific features.
 - Example: “is hexadecimal” splits into “is 0 in hexadecimal”, “is 1 in hexadecimal”, etc.
- Final layer neurons are much easier to interpret than others, because we can look at their **logit attribution**. Because the neuron's output is always a scalar times the vector `w_out`, we can compute the effect this has on the logits with the vector `w_out @ W_U`. If we think we know what the neuron represents, it *should* boost the logits for the relevant tokens.
 - This is similar to **direct logit attribution**, where we look at the direct effect of an *activation* on the logits, but here it is purely weight based. As such, we can *compare* different output tokens, but not interpret the actual logit values in an absolute sense.
 - Eg a base64 neuron should boost the tokens used in base64 strings.

- This also works somewhat for late but not final layer neurons, especially in larger models, since those *may* just be operating via how they affect the output logits.
- **Caveat:** This will not pick up on neurons that act by suppressing *incorrect* output tokens (via the inhibition of the final softmax), nor on neurons whose effect is to change the residual stream norm to affect the final LayerNorm.
- Note further that, because the MLP layers do not move information between positions, this should only boost plausible *next* tokens.
- If you *think* you understand what a neuron/direction is doing, there's a bunch of ways to validate this understanding! A rough brainstorm:
 - Look at logit attribution for the neuron, if you think it acts by directly boosting the correct logit.
 - [Ablate](#) or [resample](#) (replace with the neuron's activation on a random other prompt) the neuron on a prompt where its feature *should* be useful and see how this harms performance
 - Try [activation patching](#) between a prompt where the feature is present and one where the feature isn't, and see how much this neuron affects the output.
 - Look at the gradient of the loss with respect to the neuron activation. Compare prompts where the neuron's feature should matter to the ones where it shouldn't.
 - The neuron gradient times activation (sometimes called the neuron attribution) can be more meaningful. It's a locally linear approximation to what happens if the neuron is set to zero.
 - Train a probe to detect the relevant feature, and compare the resulting direction to the neuron.
 - Ad-hoc methods for automated testing, depending on the feature - if you can automatically detect the feature in text, then you can compare the neuron activations on text to whether the feature is present.
 - Use a Regular Expression for simple syntactic features. Works especially well with code.
 - Use software like [spaCy](#) which automatically detect linguistic features in text.
 - Use some code parsing tools (eg the Python compiler or C compiler)
 - Ask GPT-3 nicely to detect the feature

Resources

- **The main useful resource here is [Neuroscope](#)** - a website I made with a page for every neuron in various GPT-style models and the top 20 activating dataset examples
 - A tutorial on how to write [an interactive neuroscope](#) - where you can load in a model, type in text, and see how a specific neuron activates on that text.
 - This lets you automatically load in the dataset examples used in Neuroscope.
 - Currently supported models:
 - [My toy language models](#)
 - A [scan of larger SoLU models](#) up to GPT-2 Medium sized trained on a mix of python code & web text.
 - A [scan of larger SoLU models](#) trained on the Pile, up to GPT-2 Medium sized. Includes a one layer and a two layer model.
 - GPT-2
- My TransformerLens library for editing and accessing model internals - this is used to create the interactive neuroscope, and will be useful for a bunch of the more ambitious or ad-hoc ways to build a project here
 - The library lets you load in my toy language models and SoLU models
- [My mech interp explainer](#), I highly recommend looking up any unfamiliar jargon in there
- It's worth skimming [the SoLU paper](#) esp [section 6.3](#), to get a sense of what neurons exist in different parts of models and of models of different sizes.
 - I summarise these findings in [my Mech Interp Explainer](#).
- [Toy Models of Superposition](#) should build good intuitions around how to think about superposition, especially [the section on neuron superposition](#).
 - I summarise these findings in [my Mech Interp Explainer](#)
- The Captum library implements a range of attribution methods, see eg [the docs page on neuron attribution methods](#).

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- Exploring Neuroscope
 - **A* 9.1 - Just explore random neurons!** And then use the interactive neuroscope to test and verify your understanding.

- **A* 9.2** - Look for interesting conceptual neurons in the middle layers of larger models
 - Analogous to the “[numbers that refer to groups of people](#)” neuron
- **A* 9.3** - Look for examples of detokenization neurons
 - **A* 9.4** - Look for examples of trigram neurons (neurons that consistently activate on a pair of tokens, and which boost the logit of plausible next tokens)
- **A* 9.5** - Look for examples of retokenization neurons
- **A* 9.6** - Look for examples of context neurons (eg base64)
- **A* 9.7** - Look for neurons that align with any of the feature ideas listed in the next section.
- **B* 9.8** - Look for examples of neurons that have a naive (but incorrect!) initial story, and which have a much simpler explanation after further investigation
- **B* 9.9** - Look for examples of neurons that have a naive (but incorrect!) initial story, and which have a much more *complex* explanation after further investigation
- **A* 9.10** - How much does the logit attribution of a neuron align with the dataset example patterns? Does this seem consistent or totally unrelated?
 - **B* 9.11** - If you find neurons where this seems very inconsistent, can you figure out what’s going on there?
- **A-B* 9.12** - For dataset examples for neurons in a 1L network, measure how much its pre-activation value comes from the output of each attention head vs the embedding (vs the positional embedding!). If it’s dominated by specific attention heads, how much are those heads attending to the tokens you might expect?
- Seeking out specific features (note - it’s totally plausible to me that the big + important features soon split into many smaller features)
 - **Tip:** For a bunch of the below, exactly what to look for will depend heavily on how things are tokenized - I recommend exploring before deciding on precise features to look for.
 - Features that can be automatically checked for are particularly exciting!
 - **A* 9.13** - Basic syntax:
 - Start of line
 - End of sentence

- Proper noun
 - Vs start of the sentence
- Number
- Year
- Date (verify with a Regex)
- Phone numbers (verify with a Regex)
 - Country code
- <|endoftext|> token (used to indicate that two different texts were concatenated)
- Choose your own adventure!
- Linguistic features (try using [spaCy](#) to automate this!)
 - Subject of the sentence
 - Object of the sentence
 - Verb
 - Adjective
 - Indirect Object
 - Pronoun
 - Male pronoun
 - Female pronoun
 - Neutral pronoun
 - Text written in first person
 - Text written in second person
 - Text written in third person
 - Word followed by 't (can't, don't, aren't, etc)
 - Choose your own adventure!
- **A* 9.14** - Proper nouns
 - Name
 - Firstname Surname
 - Just first name
 - Just surname
 - Country
 - City

- Landmark
- Name of historical battle
- Monarch
 - Monarch of specific country
- Language
- Choose your own adventure!
- **A* 9.15** - Python Code features (models trained on the Pile will see lots of code, or my toy language/SoLU models were partly trained on Python code)
 - Variable in a function definition
 - Disambiguation features/neurons - taking a token that appears in many contexts and disambiguating this:
 - Disambiguation neuron for commas (eg in a list vs dictionary vs tuple)
 - Disambiguation neuron for colons (eg in a Python type hint vs Python for loop vs Python if statement, etc)
 - Disambiguation neuron for square bracket (eg indexing vs defining a list)
 - Disambiguation neuron for right brace (eg closing dictionary vs list)
 - **B* 9.16** - Level of indent for a line (harder because it's categorical/numeric)
 - **B* 9.17** - Level of bracket nesting (harder because it's categorical/numeric)
 - Name of variable being defined
 - Is in a comment
 - Choose your own adventure!
- **A-B* 9.18** - General code features - I'm sure there's a lot of interesting ones here! (Pile models were trained on many languages)
 - Base64 neurons
 - Hexadecimal neurons
 - Specific HTML tags
 - The closing brace for different tags
 - Detecting the programming language
 - Choose your own adventure!
- **A* 9.19** - LaTeX features (the Pile contains a lot!)
 - Common commands (\left, \right, etc)

- Section titles (\abstract, \introduction, \section, \subsection, etc)
- **A-B* 9.20** - Features in compiled LaTeX, eg paper citations (the Pile contains a lot!)
- **B* 9.21** - Any of the more abstract neurons in [Multimodal Neurons](#) (Eg Christmas, sadness, teenager, anime, Pokemon, etc - it's a wild paper!)
 - Decent chance the conceptual features only really occur in much larger models, so this may be doomed. I'd focus on the middle layers of GPT-2 XL
- Disambiguation neurons
 - **A* 9.22** - Foreign language disambiguation eg “die” in Dutch vs German vs Afrikaans (Note - a model doesn't need to have been explicitly trained on multiple languages to show this kind of thing. But I expect you to get the best results with Pile models, whose training data included eg EuroParl)
 - **A* 9.23** - Words with multiple meanings, eg bat can mean an animal or sports equipment. Can you find a separate feature for the two.
- **A* 9.24** - Search for memory management neurons (neurons with high *negative* cosine sim between w_{in} and w_{out}). What do their dataset examples look like? Is there any pattern?
- **A* 9.25** - Search for signal boosting neurons (neurons with high *positive* cosine sim between w_{in} and w_{out}). What do their dataset examples look like? Is there any pattern?
- **B-C* 9.26** - Search for neurons that clean up superposition interference.
 - One idea is neurons that correlate with some big and common feature. Eg, if many neurons represent both Python code and romance novel specific features, a single `is_python_code` neuron and a `is_romance_novel` neuron could serve to clean up all interference
 - What do their dataset examples look like? Is there any pattern?
- **A* 9.27** - Can you find split-token neurons?
 - Words are sometimes 1 vs 2 tokens depending on capitalization and whether there's a preceding space, eg “Cl|aire” vs “ Claire”. I expect the model to learn to identify the split-token case, and to create a direction similar to the “ Claire” embedding.
- **A-B* 9.28** - Can you find examples of [neuron families/equivariance](#)? (Neurons which all represent analogous and symmetric things - analogous to curves of different angles). Ideas:
 - Disambiguation neurons

- Analogous neurons for different programming languages
- Analogous neurons for different languages (English vs French vs etc)
- Neurons for past vs present vs future tense
- Neurons for different elements of a category (different digits in hexadecimal, different months, different days of the week, etc).
- Neurons which link to attention heads (eg amplifying their output, correcting them, etc):
 - **B* 9.29** - Induction should *not* trigger (eg the current token is repeated but the previous token is not, or different copies of the current string have different next tokens)
 - **B* 9.30** - Fixing a skip trigram bug
 - **A* 9.31** - This token is duplicated
 - **B* 9.32** - Splitting into token X is duplicated, for many common tokens.
- **A-B* 9.33** - Neurons which represent positional information (ie which are not invariant between position). Eg neurons that activate a bunch more on token position X (natural guesses are X=0, X=1, X=2, which are most likely to matter)
 - To detect this reliably, you'll need to input data with a random offset - if the first few tokens are always the first few tokens of a document, there will be a bunch of weird correlations!
- **B* 9.34** - What's the longest n-gram you can find that seems represented? (note - to properly count here, the neuron should be actively detecting the first element of the n gram, and to decrease its activation if that is gone)
- **B-C* 9.35** - Try training linear probes for any of the above features
 - **B-C* 9.36** - How does your ability to recover the features from the residual stream compare to MLP layer outputs vs attention layer outputs? Can you find features that can only be recovered from some of these?
 - **B-C* 9.37** - Are there features that can only be recovered from certain MLP layers?
 - **B-C* 9.38** - Are there features that are significantly easier to recover from early layer residual streams and not from later layers?
- Curiosities about neurons
 - **A* 9.39** - When you look at the max dataset examples for a specific neuron, is that neuron the most activated *neuron* on that text? What does this look like in

general?

- **A-B* 9.40** - Look at the distributions of neuron activations (both pre and post activation for GELU, and pre, mid and post for SoLU). What does this look like? How heavy tailed is it, and how well can it be modelled by a normal distribution?
 - **B* 9.41** - Do neurons vary in terms of how heavy tailed their distributions are? Does this at all correspond to monosemanticity?
 - **A* 9.42** - How similar are the distributions between SoLU and GELU?
- **A* 9.43** - What does the distribution of the LayerNorm scale and softmax denominator in SoLU look like? In particular, is it bimodal (indicating monosemantic vs highly diffuse features) or fairly smooth and unimodal?
- **B* 9.44** - Can you find any *genuinely* monosemantic neurons? That are (mostly) monosemantic across their entire activation range?
 - I expect this is best done with some automated test for whether a feature is present, eg a Regex or spaCy or code parsing.
- **B* 9.45** - Find a feature where GELU is used to calculate it in a way that ReLU couldn't be (eg, used to approximate a quadratic)
- **B* 9.46** - Can you find a feature which seems to be represented by several neurons?
 - B 9.47 - What happens to the model if you ablate *some* of these neurons? Is it robust to this, or does it need all of them to represent the feature?
- **B* 9.48** - Can you find a feature that is highly diffuse across neurons? (Ie, which is represented by the MLP layer, but which doesn't activate any particular neuron a lot)
 - I recommend verifying that the MLP matters by doing [activation patching](#) for a similar input with a different answer, or by training a probe on it.
- **B* 9.49** - Look at the [direct logit attribution](#) of neurons, and find the max dataset examples for this. How similar are these texts to the max activating dataset examples? (Direct logit attribution here means the contribution to the correct next token's logit, *weighted by neuron activation*. Ie not just weight based)
- **B* 9.50** - Look at the max *negative* direct logit attribution. Are there neurons which systematically suppress the correct next token? Can you figure out what's up with these?
- **A* 9.51** - Try comparing how monosemantic the neurons in a GELU vs SoLU model are. (Using the metric in the paper of "is there a consistent pattern in the

max activating dataset examples). Can you replicate the results that SoLU does better? What are the actual rates for each model?

- I'd start with the toy language models, where there are GELU and SoLU that are identically trained.
- **B-C* 9.52** - Can you find a better and more robust metric? How consistent is this across reasonable metrics?
- **A-B* 9.53** - The GELU and SoLU toy language models (solu-1l vs gelu-1l etc) were trained with identical initialisation and data shuffle. Can you see any correspondence between what neurons represent in each model, or is it basically random?
 - **B* 9.54** - If a feature is represented in one model, how likely is it to be represented in the other?
- **B* 9.55** - Can you find a neuron whose activation isn't significantly affected by the current token?
- Misc
 - **B-C* 9.56** - An important ability of a network will be to attend to things within the current clause or sentence. This can be approximated by only attending to nearby tokens, but can you find evidence that models are doing something more sophisticated? (Eg, they behave differently if random words are added before or after the most recent full stop). And if so, are there relevant neurons/features? (Eg directions that align more within the current clause than between clauses)
 - **C* 9.57** - Replicate [Knowledge Neurons in Pretrained Transformers](#) on a generative model (the paper uses integrated gradients to identify neurons used in factual recall). How much are these results consistent with what Neuroscope shows?
 - **A* 9.58** - Can you replicate the results of [the interpretability illusion](#) on my SoLU models, which were trained on a mix of web text and Python code? That is, find neurons that seem monosemantic on either, but with importantly different patterns.
 - I can supply the max examples on Python code, and the max examples on web text, so it should be easy enough to compare.
 - **B* 9.59** - Try doing [dimensionality reduction](#) over neuron activations across a bunch of text, and see how interpretable the resulting directions are (eg, do they have high direct logit attribution? Are there patterns in the dataset examples that maximally activate those directions? Etc). I expect this to be most interesting for

linear methods (ie not t-SNE or UMAP), which give you meaningful directions in neuron space to interpret.

- [An example from Jay Alammar](#) doing this for [Non-negative Matrix Factorisation](#) - this requires activations to be non-negative so he applies the dumb hack of setting all negative entries to zero. Another dumb hack is to create double the number of dimensions, and represent $\max(x, 0)$ in the first and $-\min(x, 0)$ in the second.
 - I think NMF will be particularly interesting because it tends to give sparse outputs
- **B-C* 9.60** - Pick a BERTology paper and try to replicate it on GPT-2! (Can vary from a partial replication of the key insights to a full replication)
 - [BERT Rediscovered the Classic NLP Pipeline](#)
 - [Visualizing and Measuring the Geometry of BERT](#)
 - Choose your own adventure: Pick anything that catches your fancy in [a primer on BERTology](#)
- **B* 9.61** - Make a PR to Neuroscope with some feature you wish it had!
- **B-C* 9.62** - Replicate the part of [Conjecture's Polytopes paper](#) where they look at the top eg 1000 dataset examples for a neuron across a ton of text and look for patterns in that
 - Is it the case that there are monosemantic bands in the neuron activation spectrum?

Previous:

200 COP in MI: Interpreting Reinforcement Learning

No comments 25 karma

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 14 Mechanistic Interpretability Quickstart Guide

2 comments, sorted by top scoring

[-] **Charlie Steiner** 3y ▼ 2 ▲ ✕ 0 ✓

Thanks a bunch for this series!

[-]

Neel Nanda

3y

▼

1

▲

✕

0

✓

Thanks! I'd be excited to hear from anyone who ends up actually working on these :)

Moderation Log

More from Neel Nanda

- 19

Building and evaluating model diffing agents

bilalchughtai, Josh Engels, Neel Nanda

6h

0
- 32

Models May Behave Worse When Eval Aware...

Senthooran Rajamanoharan, Neel ...

2d

0
- 65

models have some pretty funny attractor sta...

aryaj, Senthooran Rajamanoharan, ...

3mo

0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike’s-Eye View of ARC’s Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0